

Implementation Summary

Classes	
<pre> class Tree: def __init__(self, height): self.height = height @property def height(self): return self._height @height.setter def height(self, num): self._height = num </pre>	<pre> public class Tree { private int height; public Tree(int height) { this.height = height; } public int getHeight() { return height; } public void setHeight(int num) { height = num; } } </pre>
Python	Java
<pre> class Tree { private: int height; public: Tree(int height); int getHeight(); void setHeight(int num); }; Tree::Tree(int height) { this->height = height; } int Tree::getHeight() { return height; } void Tree::setHeight(int num) { this->height = num; } </pre>	<pre> public class Tree { public int Height { get; set; } public Tree(int height) { Height = height; } } </pre>
C++	C#

	Structs V. Classes	
	Struct	Class
C++	Default visibility of members is public	Default visibility of members is private
	Default visibility of inheritance is public	Default visibility of inheritance is private
C#	Can't be set to null	Can be set to null
	When copied, value is copied over	When copied, memory location is copied over
	Normally lives on the runtime stack	Normally lives on the heap
	Acts more as a primitive value	Is a pointer to the actual memory allocation
	Cannot inherit from other structs, but can inherit from interfaces	Can inherit from other classes and interfaces

Interfaces	
<pre> from abc import ABC, abstractmethod class File: @abstractmethod def readFile(self, path): pass class TxtFile (File): def readFile(self, path): # read text file </pre>	<pre> interface File { public String ReadFile(String path); } class TxtFile implements File { @Override public String ReadFile(String path) { // read text file } } </pre>
Python	Java
<pre> class File { public: virtual string readFile(string path) = 0; }; class TxtFile : File { public: string readFile(string path) override { // read text file } }; </pre>	<pre> public interface IFile { public string ReadFile(string path); } public class TxtFile : IFile { public string ReadFile(string path) { // read text file } } </pre>
C++	C#

Enums	
<pre> from enum import Enum class Colors(Enum): RED = '#FF0000' GREEN = '#00FF00' BLUE = '#0000FF' </pre>	<pre> // simple enum Colors { RED, GREEN, BLUE } // complex enum Colors { RED("#FF0000"), GREEN("#00FF00"), BLUE("#0000FF"); private String hexStr; Colors(String hexStr) { this.hexStr = hexStr; } } </pre>
Python	Java
<pre> // simple enum class Colors { RED, GREEN, BLUE }; // complex enum class Colors : unsigned int { RED = 0xFF0000, GREEN = 0x00FF00, BLUE = 0x0000FF }; </pre>	<pre> // simple public enum Colors { RED, GREEN, BLUE } // complex public enum Colors : uint { RED = 0xFF0000, GREEN = 0x00FF00, BLUE = 0x0000FF }; </pre>
C++	C#

Tuples	
<pre># simple simple = (5, 3.1) print(simple[0]) print(simple[1]) # named from collections import namedtuple Named = namedtuple('Named', ['x', 'y']) named = Named(5, 3.1) print(named.x) print(named.y)</pre>	<pre>// no native support</pre>
Python	Java
<pre>#include <tuple> #include <iostream> using namespace std; int main() { // simple tuple<int, float> simple(5, 3.1f); cout << get<0>(simple) << endl; cout << get<1>(simple) << endl; // named struct Named { int x; float y; }; Named named { 5, 3.1f }; cout << named.x << endl; cout << named.y << endl; }</pre>	<pre>using System; public class Tuples { public static void Main(string[] args) { // simple Tuple<int, float> simple = new(5, 3.1f); Console.WriteLine(simple.Item1); Console.WriteLine(simple.Item2); // named (int x, float y) named = new(5, 3.1f); Console.WriteLine(named.x); Console.WriteLine(named.y); } }</pre>
C++	C#

Concept Summary

Class	Struct	Abstract Class	Interface	Enum	Record / Dataclass	Tuple	Named Tuple
General UDT.	In C++, only difference is default visibility. In C#, used as a kind of lightweight class to move around data and clone its values on assignment.	When one or more methods cannot be implemented. You can create fields and treat it as a super class. Useful for binding several sub classes to a single super class type and creating methods that they must implement. Note that in Java and C#, you can only inherit from one abstract class.	When all methods cannot be implemented. Cannot have fields (except for constants). A class can inherit from multiple interfaces.	When you want a data type with a small collection of valid values. In C++ and C#, enum values are bound to ints. In Python, you can bind to whatever data you want. In Java, you can create multiple fields for each enum value to have.	When you want immutable fields, a check for equality, a string representation of the object, and are mostly concerned with the data held in the UDT but not so much concerned with its methods.	Ad-hoc UDT for passing data around.	Assigns names to tuple values.